
RAPPORT DE STAGE

AREZKI LITICIA

DOUBLE LICENCE MATHÉMATIQUES ET INFORMATIQUE
LICENCE 3

**Moteur de recherche sémantique multimodal (Image-Texte)
basé sur CLIP**
Machine Learning / Data Science

01 Juin 2026 - 15 Juillet 2026

Tuteurs académiques :

M. PIERRE ROUSSELIN
roussein@math.univ-paris13.fr
M. CHARLES DE CLERCQ
declercq@math.univ-paris13.fr

Tuteur Laboratoire :

M. LAZHAR LABIOD
lazhar.labiod@u-paris.fr

Résumé :

Ce stage de 6 semaines, réalisé au Centre Borelli (UMR 9010) de l'Université Paris Cité, avait pour objectif de concevoir un moteur de recherche sémantique multimodal permettant de retrouver des images à partir de descriptions en langage naturel, ou d'images similaires à partir d'une image requête.

Le système développé repose sur CLIP (Contrastive Language-Image Pre-training), un modèle d'OpenAI qui projette textes et images dans un espace vectoriel commun. L'implémentation, réalisée en Python avec PyTorch et Hugging Face Transformers, comprend deux modules : la recherche texte vers image et la recherche image vers image, testés sur une base de données locale.

Le travail a consisté à extraire les embeddings d'une base d'images, à encoder les requêtes textuelles, puis à implémenter une recherche par similarité cosinus. Les résultats sont ensuite visualisés de façon simple pour évaluer la pertinence du système.

Ce stage a permis de mettre en pratique des notions d'apprentissage profond multimodal, de représentation vectorielle et de recherche d'information, en s'appuyant sur des outils modernes.

Remerciements :

À mon tuteur de laboratoire : M. Lazhar LABIOD, Maître de conférences à l'Université Paris Descartes

Je tiens à vous exprimer ma sincère et profonde gratitude de m'avoir donné l'opportunité de réaliser ce stage sous votre supervision. Votre aide, vos conseils et votre patience ont été une source de motivation constante et ont rendu cette expérience particulièrement enrichissante.

À mon tuteur académique : M. Charles DE CLERCQ, Responsable de la troisième année Double Licence

Je vous remercie chaleureusement pour votre soutien ainsi que pour vos encouragements, votre écoute, votre disponibilité, et la confiance que vous m'avez accordée, même dans les périodes les plus difficiles de mes études. Votre bienveillance m'a donné la confiance nécessaire pour aller au bout de ce parcours.

À mon responsable de formation : M. Pierre ROUSSELIN, Responsable de la Double Licence

Merci de m'avoir accueilli au sein de la Double licence Mathématiques et Informatique, merci pour votre accompagnement depuis mon premier jour à l'Université Sorbonne Paris Nord. Tout au long de ces trois années, votre suivi, votre exigence et la passion avec laquelle vous transmettez votre savoir m'ont énormément inspiré.

Je souhaite également adresser mes remerciements à ma famille, mes proches pour leur aide dans ma recherche de stage et leurs encouragements tout au long de cette année.

Pour finir, je remercie mes camarades de promotion pour les encouragements et la bonne ambiance qui ont accompagné cette fin d'année universitaire.

AREZKI Leticia

Sommaire

Sommaire	3
1. Introduction	4
2. Présentation du laboratoire	5
3. Présentation du sujet de stage	5
3.1 Contexte et problématique	5
3.2 Le modèle de fondation CLIP	6
3.2.1 CLIP	6
3.2.2 L'apprentissage constructif et l'espace latent	6
3.3 Objectif et missions du stage	7
4. Contribution et réalisation technique	8
Architecture du moteur de recherche multimodal	8
4.1 La préparation des images	8
4.2 La recherche et le calcul de similarité	9
4.4 Conception de l'interface utilisateur graphique	11
4.5 L'extension à la recherche Image vers Image (une option supplémentaire)	12
5. Evaluation, Résultat et Limites	12
5.1 Présentation du jeu de données	12
5.2 Analyse des performances et complexité	13
5.3 Exemples de requêtes et observations	13
5.4 Limites du modèle identifiées	15
5.5 Intégration de FAISS et de l'interface utilisateur	16
5.6 Perspectives d'amélioration futures	17
6. Code	17
6.1 Le code principale avec Tkinter et FAISS :	17
6.2 La bibliothèque utilisée	25
7. Conclusion	28
8. Bibliographie	29
8.1. Articles Scientifiques	29
8.2. Documentation Techniques et Liens Web	29

1. Introduction

Dans le cadre de ma troisième année de Double Licence Mathématiques et Informatique à l'Université Sorbonne Paris Nord, j'ai choisi d'orienter mon parcours vers la science des données et l'intelligence artificielle, des domaines où la synergie entre la rigueur mathématique et le développement logiciel prend tout son sens. C'est avec l'objectif d'appliquer ces compétences théoriques à des problématiques concrètes de recherche que j'ai effectué un stage de six semaines au sein du Centre Borelli (UMR 9010) à l'Université Paris Cité.

Le Centre Borelli est un laboratoire de recherche connu pour ses travaux interdisciplinaires, notamment en mathématiques appliquées et en traitement des données. Intégrer cette structure m'a permis de m'immerger dans un environnement scientifique stimulant et de me confronter aux défis actuels de l'apprentissage profond (deep learning).

Mon stage s'est articulé autour d'une problématique centrale et en pleine expansion : la recherche d'information multimodale. Avec la multiplication exponentielle des données numériques, l'accès à l'information visuelle reste un défi majeur lorsque les images ne disposent d'aucune description textuelle préalable. L'enjeu de mon projet a donc été de répondre à la question suivante : **comment permettre la recherche d'images performante dans une base non annotée à partir d'une requête en langage naturel ?**

Pour y répondre, nous nous sommes appuyés sur le modèle CLIP (Contrastive Language-Image Pre-training), une architecture développée par OpenAI capable de lier le texte et l'image au sein d'un même espace latent vectoriel. Mon travail a consisté à appréhender le fonctionnement de ce modèle, à constituer un jeu de données thématique, puis à extraire et stocker les représentations vectorielles (embeddings) des images. J'ai ensuite implémenté un système de calcul de similarité cosinus pour faire correspondre les requêtes textuelles des utilisateurs aux images de la base, avant de concevoir une interface de visualisation simple pour restituer les résultats.

Ce rapport présente la démarche méthodologique adoptée, les choix techniques effectués (notamment l'utilisation de PyTorch, Python, Hugging Face ...) ainsi qu'une analyse critique des résultats obtenus. Enfin, cette expérience au cœur de la recherche d'information et des systèmes de Machine Learning constitue un tremplin idéal pour la suite de mon parcours.

2. Présentation du laboratoire

Le Centre Borelli (UMR 9010) est un laboratoire de recherche sous la cotutelle principale de l'Université Paris Cité, du CNRS, de l'École Normale Supérieure Paris-Saclay et du Service de Santé des Armées. Dirigé par M. Nicolas Vayatis, le laboratoire est implanté au cœur de la communauté scientifique parisienne, notamment sur le campus de l'Université Paris Cité dans le 6^e arrondissement. Cette structure se distingue particulièrement par sa forte culture interdisciplinaire, où les mathématiques appliquées s'allient aux sciences du numérique pour résoudre des problématiques concrètes, médicales et sociétales.

Les activités de recherche du Centre Borelli s'organisent autour de plusieurs axes majeurs, notamment le traitement d'images et de signaux, la vision par ordinateur, le Machine Learning et la modélisation mathématique du vivant. Le laboratoire est particulièrement reconnu pour sa capacité à faire le pont entre la théorie mathématique fondamentale et le développement d'algorithmes appliqués à des données massives.

C'est au sein de l'équipe spécialisée en science de données et apprentissage automatique que j'ai intégré la structure pour travailler sur les modèles de fondation multimodaux. Le laboratoire offre un cadre idéal pour ce type de projet, disposant à la fois de l'expertise scientifique nécessaire pour appréhender les fondements mathématiques des modèles de deep learning, et des ressources de calcul indispensables au traitement des représentations vectorielles à grande échelle.

3. Présentation du sujet de stage

3.1 Contexte et problématique

Avec l'explosion de la production de données numériques, l'accès rapide et pertinent à l'information visuelle est devenu un enjeu technologique majeur. Traditionnellement, la recherche d'image repose sur des métadonnées ou des annotations textuelles préalablement saisies à la main. Cependant, face à des volumes massifs de données, cette indexation manuelle devient impossible à réaliser.

La problématique centrale de ce stage consiste à répondre à la question suivante : Comment permettre la recherche d'images dans une base non annotée à partir de descriptions en langage naturel ?

Pour lever ce verrou technologique, le projet s'inscrit dans le domaine de l'apprentissage multimodal. Contrairement aux approches classiques du deep learning qui traitent le texte et l'image séparément, l'objectif est d'exploiter des modèles capables de lier ces différentes modalités de données.

3.2 Le modèle de fondation CLIP

3.2.1 CLIP

Le système développé durant ce stage repose sur l'utilisation du modèle **CLIP** (Contrastive Language-Image Pre-training), une architecture de pointe introduite par OpenAI en 2021.

Le principe fondamental de CLIP est d'apprendre un espace latent commun où les descriptions textuelles et les images deviennent directement comparables. Le modèle est composé de deux encodeurs distincts :

- Un encodeur visuel (image encoder) qui convertit une image en un vecteur caractéristiques.
- Un encodeur textuel (text encoder) qui convertit une phrase en un vecteur dans ce même espace.

3.2.2 L'apprentissage constructif et l'espace latent

La force de CLIP réside dans sa méthode d'entraînement, appelée **apprentissage contrastif**. Contrairement aux classifieurs classiques entraînés à prédire une étiquette fixe parmi une liste prédéfinie (comme le jeu de données ImageNet), CLIP est entraîné sur des centaines de millions de paires (image,texte) récoltées sur internet.

Le but de cet entraînement est de maximiser la colinéarité des représentations vectorielles des paires légitimes tout en minimisant celle des paires incorrectes. Mathématiquement, si l'on considère un lot de N paires d'images et de textes, le modèle génère une matrice de similarité de taille : $N \times N$

- Sur la diagonale principale se trouvent les N paires réelles (L'image i correspond au texte i). Le modèle cherche à maximiser le produit scalaire de ces vecteurs.
- En dehors de la diagonale se trouvent les $N^2 - N$ paires incorrectes (l'image i avec un texte $j \neq i$). Le modèle cherche à minimiser leur produit scalaire

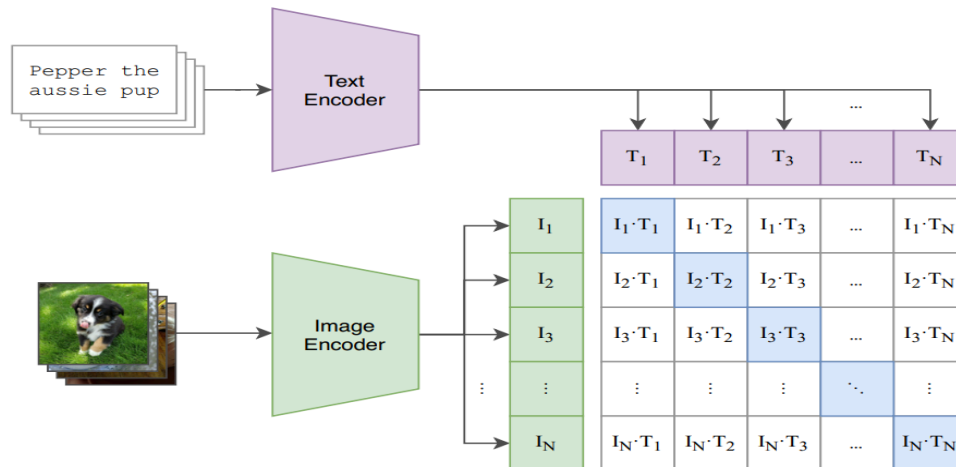


Figure 1 - Matrice de similarité $N \times N$ dans l'apprentissage contrastif de CLIP

Pour optimiser simultanément l'encodeur de texte et l'encodeur d'image, CLIP utilise une fonction de perte de type entropie croisée symétrique (symmetric cross-entropy loss) appliquée aux scores de similarité.

Grâce à cette approche globale, le modèle acquiert une compréhension sémantique fine et flexible. Une phrase et une image décrivant la même scène se retrouveront géométriquement très proches dans cet espace vectoriel, ce qui permet d'effectuer des tâches d'évaluation à zéro exemple et de s'affranchir du besoin d'étiquetage manuel des données.

3.3 Objectif et missions du stage

L'objectif principal de ce stage est de concevoir et de développer un moteur de recherche sémantique multimodal fonctionnel. Contrairement aux autres moteurs de recherche classiques basés sur des mots clés exactes (requêtes textuelle strictes), ce système doit être capable de comprendre le sens profond d'une requête utilisateur (le contexte, les concepts abstraits) et de lui associer l'image la plus pertinente au sein d'une base de données volumineuse.

Pour mener à bien ce projet, plusieurs missions successives et complémentaires m'ont été confiées :

- **Étude bibliographique et prise en main des outils** : Comprendre les fondements théoriques du Deep Learning multimodal, étudier l'architecture du modèle *CLIP* avec l'utilisation de *PyTorch* et les bibliothèques *d'Hugging Face*.

- **Conception du pipeline d'indexation** : Développer un script capable de charger une base de données d'images, de prétraiter ces images et d'extraire leurs caractéristiques sous forme de vecteurs d'embeddings afin de les stocker d'une manière persistante.
- **Développement de la logique de la recherche** : Implémenter l'encodage des requêtes textuelles et l'algorithme de calcul similarité pour faire correspondre le texte et les images.
- **Optimisation** : adapter le code initial pour faire face aux contraintes réelles du laboratoire (traitement des bases avec un très grand nombre d'images). Cela implique de restructurer le code pour passer d'un traitement séquentiel vers un traitement de lots, d'optimiser la gestion de la mémoire RAM.
- **La réalisation d'une visualisation simple des résultats** : analyse des résultats et le bon affichage.

4. Contribution et réalisation technique

Ce chapitre est consacré à la mise en œuvre pratique de ce moteur de recherche sémantique multimodal. Après avoir étudié les concepts théoriques du modèle CLIP, il a fallu concevoir l'architecture globale du système et traduire ces mécanismes en scripts Python fonctionnels.

Architecture du moteur de recherche multimodal

Le moteur de recherche fonctionne en deux grandes étapes : une étape de préparation d'images (ce qui se fait à l'avance) et une étape de recherche (quand l'utilisateur tape un texte).

4.1 La préparation des images

Avant de pouvoir faire une recherche, l'ordinateur doit analyser toutes les images pour comprendre ce qu'elles contiennent. C'est le rôle de la fonction `extraction_vecteur_img()` :

- Lecture des images une par une et les prépare pour le modèle CLIP
- La transformation des images en vecteurs de 512 coordonnées qui présente la carte d'identité mathématique qui résume le sens de l'image, d'une façon où chaque coordonnée présente : une couleur , une taille ... de l'image et l'ensemble de toutes ses coordonnées forme une image dans l'espace visuel, et forme un vecteur dans l'espace vectoriel
- Sauvegarde de tous les vecteurs de chaque image dans un fichier texte où on associe chaque image à son vecteur afin d'éviter de les calculer à chaque fois.

4.2 La recherche et le calcul de similarité

Quand un utilisateur tape une phrase, le programme cherche l'image qui ressemble le plus à cette phrase. J'ai créé deux méthodes différentes pour faire cette recherche :

- ❖ **La méthode vecteur - vecteur** : l'utilisateur donne un texte. Le programme transforme ce texte en un vecteur (son embedding vecteur), puis il ouvre le fichier de fiches d'identité des images. Il compare le texte avec la première image, puis la deuxième, et ainsi de suite jusqu'à la fin du fichier avec une boucle **for**, en utilisant le **cosinus similarité**, qui calcule le cosinus entre les deux vecteurs, et à chaque fois on garde le cosinus le plus grand. C'est une méthode simple qui compare chaque image toute seule mais qui coûte beaucoup et qui devient lente avec plusieurs images (pour plus de 6000).

La formule utilisée pour le calcul de la similarité cosinus :

$$\text{Cosinus similarité} = (A \bullet B) / (||A|| \times ||B||)$$

Ou :

- A et B sont des vecteurs
- $A \bullet B$ est le produit scalaire des vecteurs A et B
- $||A||$ c'est la norme de A et $||B||$ c'est la norme de B

- ❖ **La méthode matricielle** : Pour aller plus vite et pour bien utiliser le principe de CLIP, ce programme rassemble toutes les fiches d'identité des images dans **une matrice** de $N \times 512$ dimensions (en utilisant un nombre N d'image). Ensuite, au lieu de comparer le texte avec chaque image en même temps. Le programme fait un **produit matriciel**. Cela permet de comparer le texte avec toutes les images en même temps. Le programme normalise le produit en **un seul produit scalaire global** et il renvoie la ligne qui a le plus grand score.

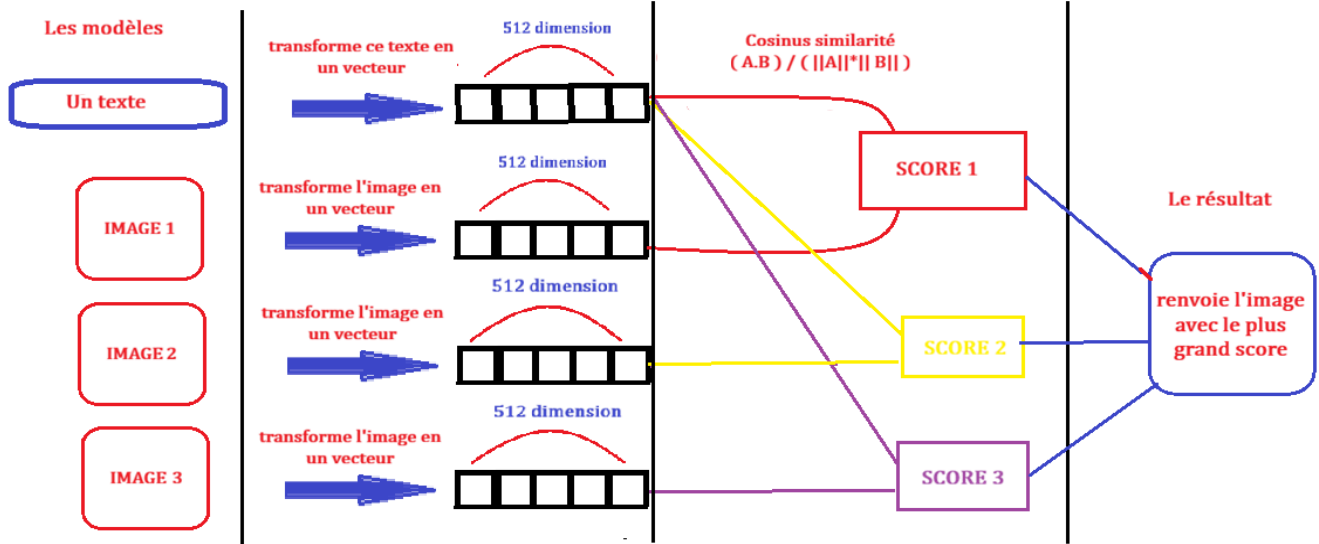


Figure 2 - Méthode 1 : vecteur - vecteur (en prenant l'exemple de N=3)

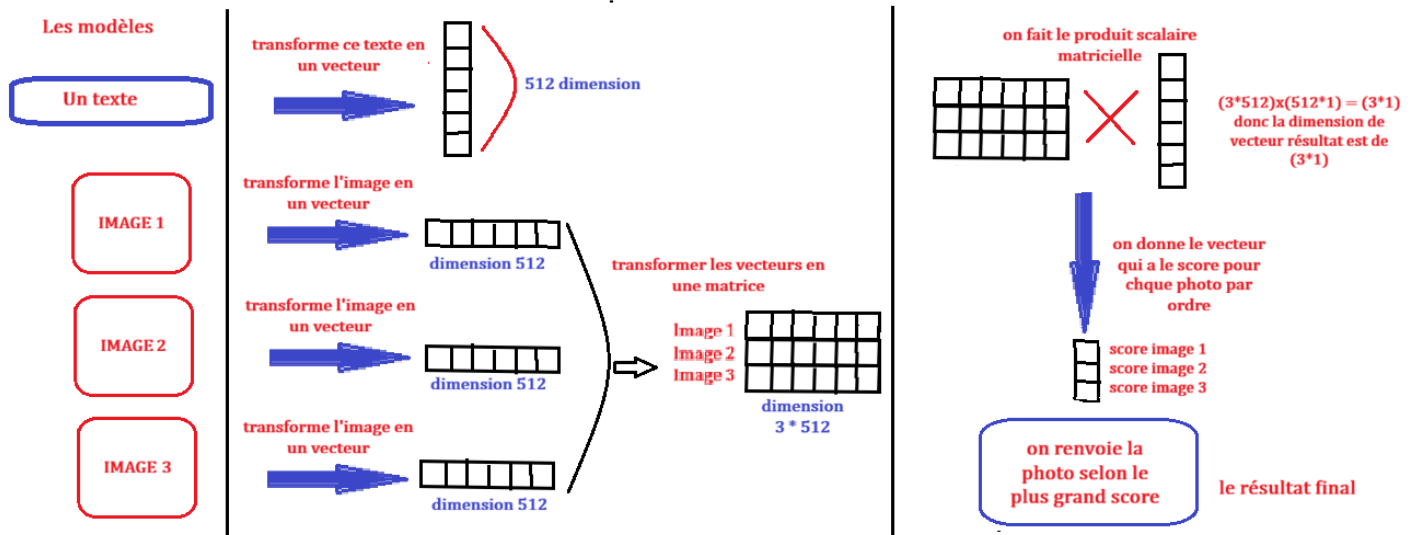


Figure 3 - Méthode 2 : La matrice (en prenant l'exemple de N=3)

- ❖ **La méthode d'indexation avec FAISS** : Pour dépasser les limites de la méthode matricielle sur de très grands volumes de données et supprimer l'usage de boucles de

comparaison ou de produits matriciels bruts sous PyTorch, nous avons intégré la bibliothèque **FAISS** (Facebook AI Similarity Search).

Dans cette configuration, les vecteurs d'images précalculés de dimension $D = 512$ sont convertis en tableaux NumPy (float32) et normalisés selon la norme de L_2 . Nous étudions ensuite un index de type **IndexFlatIP** (produit scalaire). Grâce à la normalisation préalable des vecteurs d'images et du vecteur de requête textuelle, la recherche par produit scalaire (**Inner Product**) équivaut mathématiquement au calcul de la similarité cosinus:

$$\text{Similarité Cosinus}(A, B) = A \cdot B \quad \text{car } \|A\| = \|B\| = 1$$

L'index FAISS réalise la recherche des k proches voisins en parallèle de manière extrêmement optimisée. Cette approche permet d'obtenir directement les scores et les indices des images les plus pertinentes sans aucune boucle Python.

4.4 Conception de l'interface utilisateur graphique

Afin de rendre le moteur de recherche sémantique plus accessible et interactif, une application graphique complète a été développée en Python à l'aide de la bibliothèque **Tkinter**. L'interface adopte une charte graphique sombre (dark mode) structurée pour simplifier l'expérience utilisateur :

- **Barre de recherche sémantique** : Permet de saisir la description textuelle en langage naturel. Un mécanisme de gestion du placeholder dynamique pour guider l'utilisateur.
- **Paramètre de sélection k** : Un champ dédié permet de définir précisément le nombre de résultats souhaités. Par défaut, il renvoie une image.
- **Grille résultat adaptative** : Les images s'organisent automatiquement en colonnes (de 1 à 4) selon la largeur de la fenêtre. Chaque carte d'image présente l'image d'origine redimensionnée (via Pillow) et affiche dynamiquement le rang du résultat ainsi que le pourcentage de correspondance sémantique calculé.
- **Asynchronisme et Multithreading** : Pour éviter que l'interface graphique ne se fige pas pendant les phases lourdes (chargement de CLIP, extraction initial des vecteurs ou calculs FAISS), toutes ces tâches de calcul sont déportées dans des threads d'exécution séparés (threading.Thread). L'interface utilisateur reste ainsi fluide et réactive à chaque instant.
- **Le programme fonctionne en boucle** jusqu'à ce que l'utilisateur décide de quitter l'application.

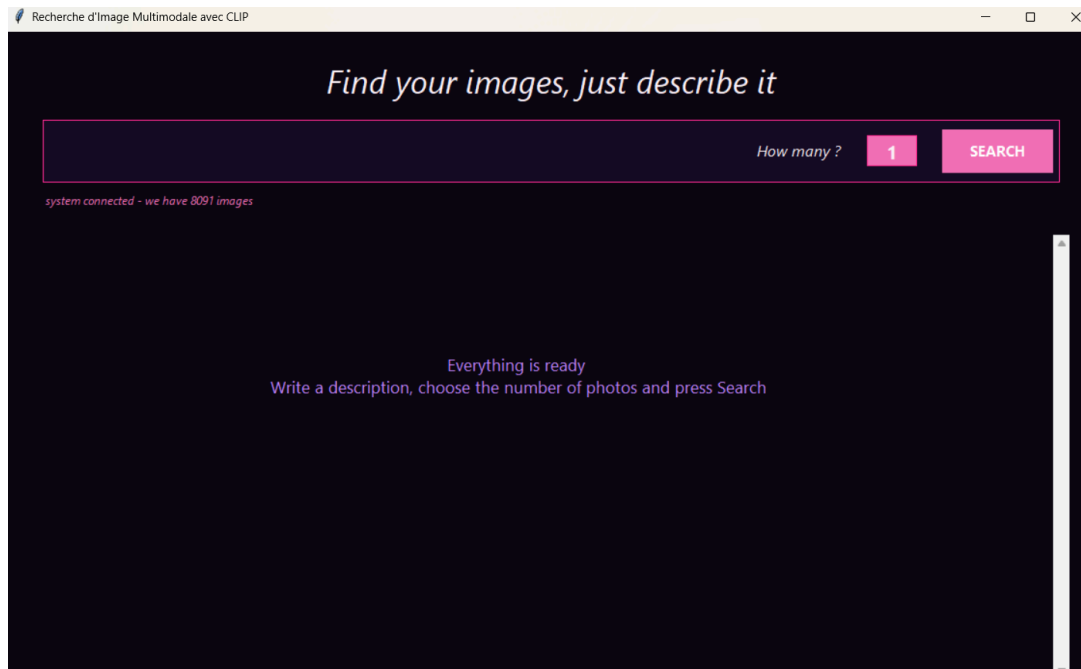


Figure 4 : interface graphique de l'application

4.5 L'extension à la recherche Image vers Image (une option supplémentaire)

Outre la recherche à partir d'une requête textuelle, l'architecture développée permet également d'effectuer une recherche d'images similaires à partir d'une image fournie par l'utilisateur (requête visuelle)

Le mécanisme s'appuie sur la même cohérence géométrique de l'espace latent de CLIP :

1. L'utilisateur fournit une image requête
2. Cette image passe par l'encodeur visuel (image encoder) via la fonction [get_embd_img_direct\(\)](#) pour extraire son embedding de dimension 512.
3. Ce vecteur normalisé est multiplié par la matrice globale de la base de données
4. Le système retourne l'image ayant obtenu le score de similarité cosinus le plus élevé

5. Evaluation, Résultat et Limites

5.1 Présentation du jeu de données

Afin d'évaluer les performances et la rapidité du moteur de recherche sémantique développé, nous avons utilisé la base de données ouverte **Flickr8k**. Ce jeu de données est un standard dans le domaine

de l'apprentissage multimodal, regroupant précisément 8092 images qui décrivent diverses scènes du quotidien, des actions, des personnes ou des animaux dans des environnements différents.

5.2 Analyse des performances et complexité

Lors de l'implémentation de la méthode matricielle, une analyse mathématique de la complexité a été menée. L'utilisation des opérations de tenseurs sur PyTorch permet une parallélisation massive des calculs. Pour une base de données contenant N images et des embeddings de dimension D ($D=512$), le produit matrice vecteur requiert $N \times D$ opérations de multiplication et d'addition. La complexité Algorithmique est donc en $O(N \times D)$. L'optimisation réside dans l'utilisation de la structure de données matricielle (GPU) par rapport à une boucle séquentielle en $O(N)$ exécutée de manière isolée en Python.

5.3 Exemples de requêtes et observations

Dans chaque exemple on donne un texte ou une description et le moteur renvoie le nom de l'image et son cosinus et puis il ouvre l'image dans une nouvelle fenêtre.

Exemples avant la réalisation de l'interface graphique :

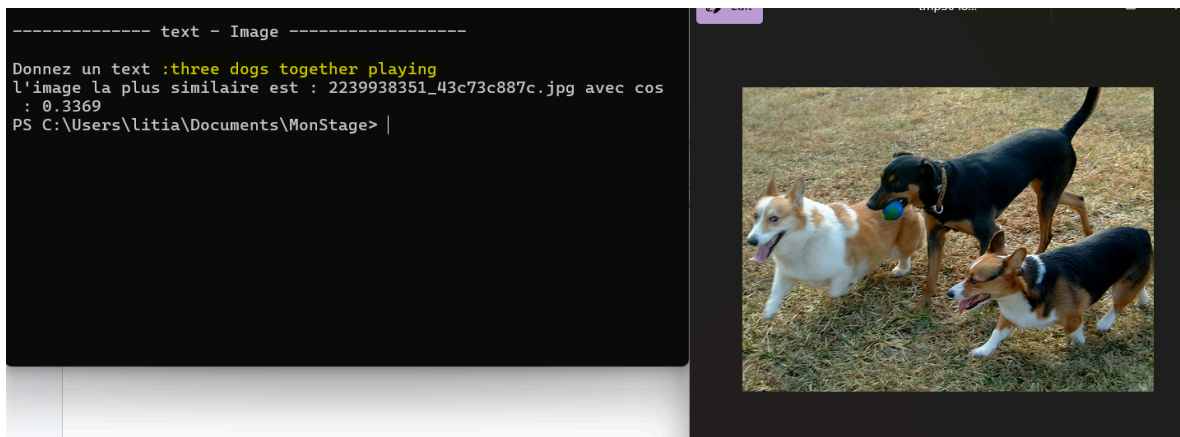


Figure 5 : Exemple avec une requête en anglais (un nombre exact de trois chiens)

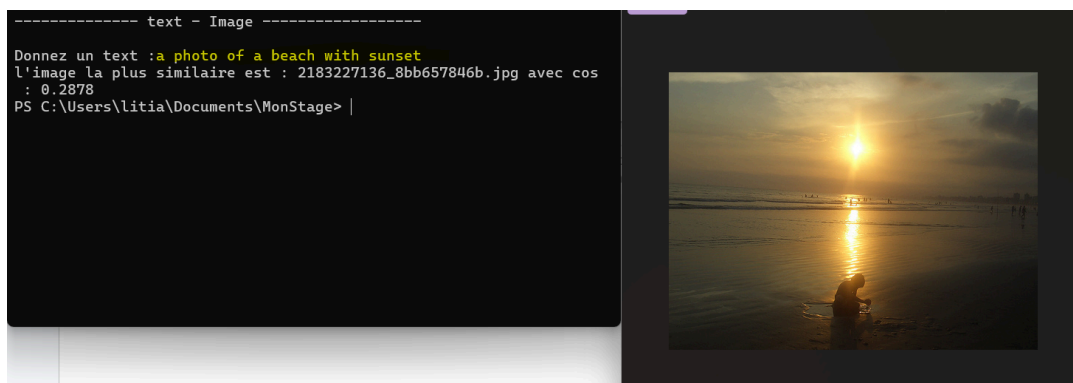


Figure 6 : Un lieu

Exemples après la réalisation de l'interface graphique :

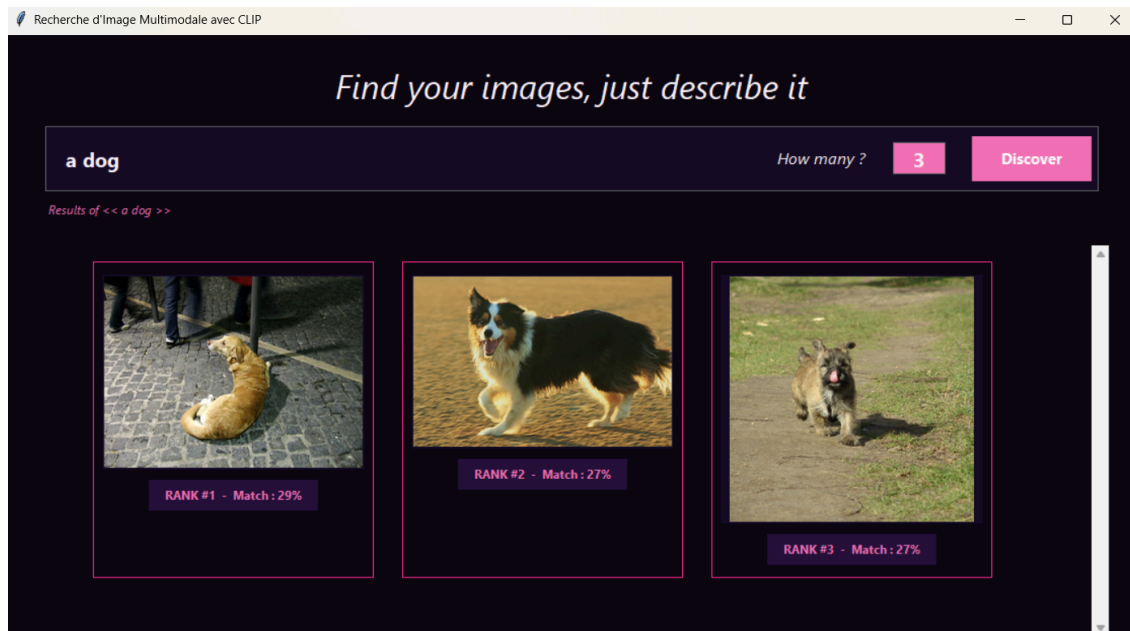


Figure 7 : Exemple avec une requête en anglais avec 3 images

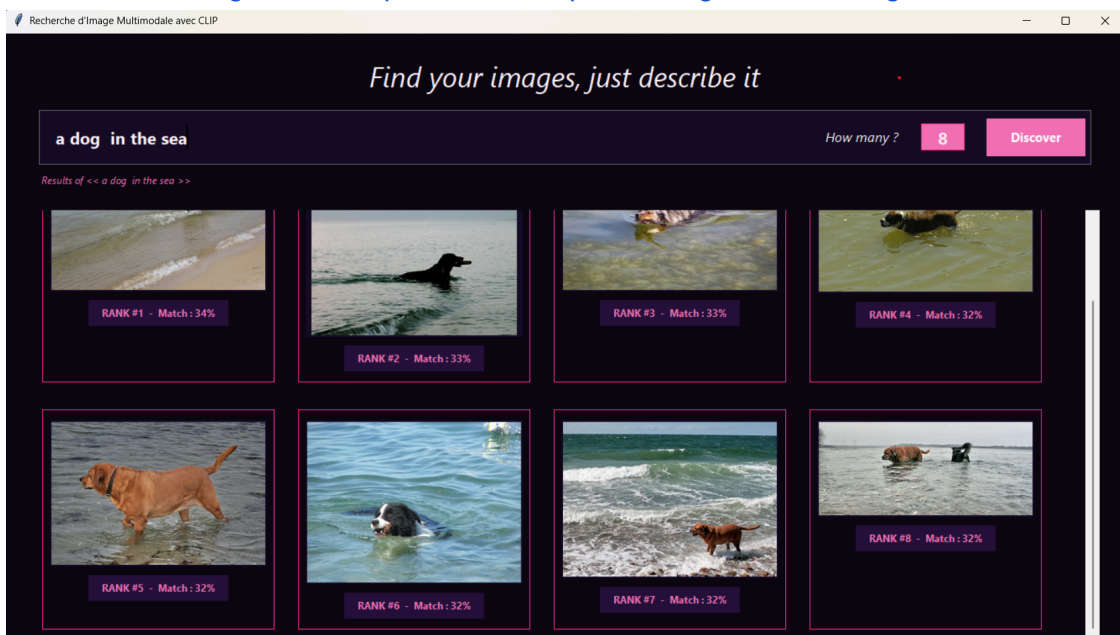


Figure 8 : Exemple avec une requête détaillée avec 8 images

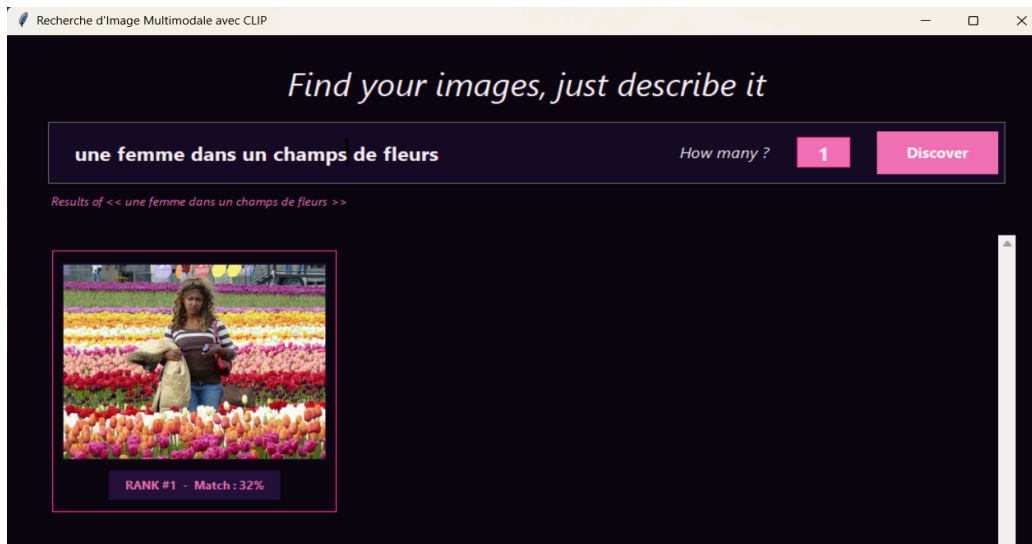
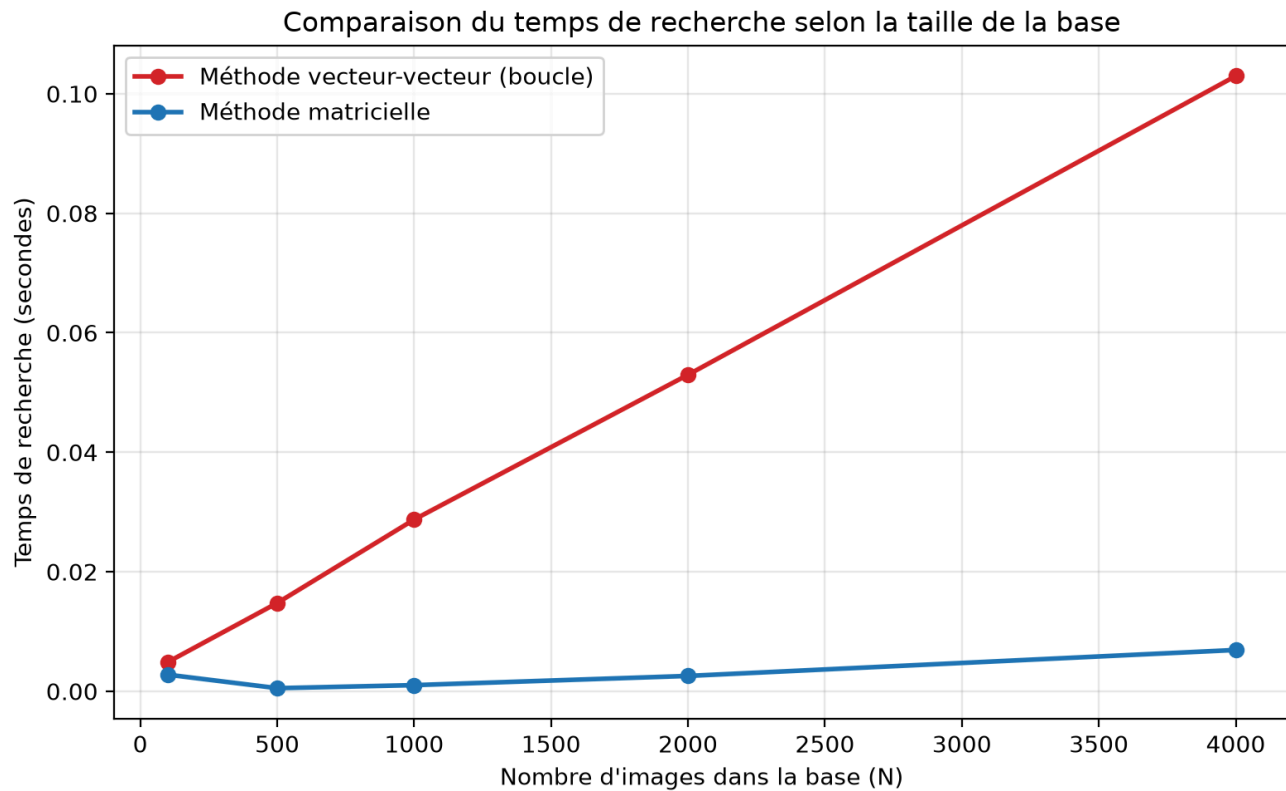


Figure 9 : Exemple avec une requête en français détaillée avec un nombre d'image 1

5.4 Limites du modèle identifiées

L'utilisation pratique de l'application met en évidence deux contraintes temporelles importantes lors de son initialisation :

- **Le chargement du modèle CLIP** : Lors du lancement du programme, cela prend environ 5 secondes pour se lancer, car le modèle doit instancier des millions de paramètres.
- **L'extraction et le chargement des embeddings** : Lors de la première utilisation de notre base de données, le traitement initial qui consiste à parcourir les 8092 images pour extraire leurs vecteurs caractéristiques est une opération lourde (elle prend environ 12 minutes), mais une fois ces vecteurs sauvegardés dans un fichier vecteurs_images.pth, leur chargement est très rapide et ne prend pas beaucoup de temps (il prend environ 0.5 seconde).
- **La dépendance au contenu de la base de données** : Le moteur de recherche ne génère pas de nouvelles images, il se contente de calculer la plus grande proximité géométrique dans l'espace latent parmi les images disponibles. Par conséquent, si l'utilisateur soumet une requête très précise comme "un chien qui court", mais la base de données ne contient aucune photo d'un chien, le modèle retournera l'image la plus proche sémantiquement dans votre espace vectoriel. On risque alors d'obtenir la photo d'un chat par exemple. Le système dépend alors totalement de la richesse et de la diversité du jeu de données initial.



5.5 Intégration de FAISS et de l'interface utilisateur

L'intégration de la bibliothèque FAISS et le développement de l'interface graphique sous Tkinter ont permis de finaliser le projet en répondant aux contraintes de performance et d'ergonomie :

- **Performance de recherche :** L'indexation par produit scalaire de FAISS combinée à la normalisation des descripteurs a permis d'éliminer les boucles de calcul manuelles en Python. L'extraction des k meilleurs résultats sur la base *Flickr8k* est désormais instantanée.
- **Visualisation et expérience utilisateur :** L'interface graphique a permis d'encapsuler la complexité des modèles de Deep Learning (CLIP). Grâce à l'asynchronisme (multithreading), l'application reste réactive lors du chargement des modèles et de l'indexation, offrant un rendu fluide des résultats sous forme de grille dynamique

5.6 Perspectives d'amélioration futures

Bien que le prototype actuel soit pleinement fonctionnel, performant et doté d'une interface graphique, plusieurs axes d'amélioration peuvent être envisagés pour étendre ce travail :

- **Indexation de bases à très grande échelle** : Pour des millions d'images, l'utilisation d'index de type IndexFlatIP (recherche exacte) peut devenir gourmande en mémoire. On pourrait explorer des index de quantification vectorielle pour effectuer des recherche ultra-rapide
- **Déploiement Web** : Remplacer l'interface Tkinter par une application Web plus moderne à l'aide de frameworks comme Streamlit, facilitant ainsi l'accès distant au moteur de recherche.
- **Recherche Images-Image** : améliorer l'interface pour une option supplémentaire, en plus de la requête textuelle, l'utilisateur pourra charger une image et chercher des images similaires.

6. Code

6.1 Le code principale avec Tkinter et FAISS :

Dans un main on a le programme suivant en Python avec Tkinter (affichant les k meilleures photos) en utilisant la méthode matricielle avec FAISS

```
import bib
import os
import torch
import threading
import faiss
import numpy as np
import tkinter as tk
from tkinter import ttk, messagebox
from PIL import Image, ImageTk

# ---- Configuration de la charte graphique globale de l'interface ----

COLOR_BG = "#0A0512"
COLOR_CARD = "#140A24"
COLOR_PRIMARY = "#F472B6"
COLOR_PRIMARY_HOVER = "#D91B75"
COLOR_ACCENT = "#A855F7"
```

```

COLOR_ACCENT_BG = "#25113B"
COLOR_TEXT_MAIN = "#FDF2F8"
COLOR_TEXT_MUTED = "#C084FC"
COLOR_BORDER = "#FF2E93"
COLOR_BORDER_ACTIVE = "#10B981"
COLOR_ERROR = "#FF4B4B"

# Définition de chemin d'accès au fichier des descripteurs
FICHIER_PTH = "vecteurs_image.pth"

class ImageSearchAPP(tk.Tk) :
    def __init__(self) :
        #initialisation et configuration géométrique de la fenêtre principale
        super().__init__()
        self.title("Recherche d'Image Multimodale avec CLIP")
        self.geometry("1000x820")
        self.minsize(800,600)
        self.configure(bg=COLOR_BG)

        # Attributs d'état de l'application
        self.embeddings = None
        self.image_names = []
        self.cashed_photos = []
        self.last_results = []
        self.resize_debounce_id = None
        self.index = None
        self.dim_embedding = None
        self.setup()
        self.load_embeddings()

    def setup (self) :
        #Conteneur supérieur dédié aux paramètre et à la barre de recherche
        top_panel = tk.Frame ( self,bg=COLOR_BG,padx=40,pady=25)
        top_panel.pack(fill="x",side="top")

```

```

        #Label de titre de l'application
        title = tk.Label( top_panel, text="Find your images, just describe
it",fg = COLOR_TEXT_MAIN , bg = COLOR_BG , font = ("Segoe UI",25,"italic"))
        title.pack(pady=(0,15))
        search = tk.Frame ( top_panel, bg = COLOR_CARD ,
highlightbackground=COLOR_BORDER, highlightthickness=1 )
        search.pack(fill="x",ipady=3)

        # champs de saisie du texte de l'utilisateur (Touche Entrée)
        self.search_entry = tk.Entry ( search,bg=COLOR_CARD,fg =
COLOR_TEXT_MUTED , bd = 0 , highlightthickness=0 ,
highlightbackground=COLOR_PRIMARY,font=("Segoe UI",15,"bold"))

self.search_entry.pack(side="left",fill="x",expand=True,padx=18,pady=10)
        self.search_entry.insert(0,"")
        self.search_entry.bind("<Return>", lambda e : self.handle_search())
        self.search_entry.bind("<FocusIn>", self.clear_placeholder )
        self.search_entry.bind("<FocusOut>", self.restore_placeholder)


        # Module de configuration du paramètre de voisinage K
        k_param = tk.Frame(search,bg=COLOR_CARD,padx=15)
        k_param.pack(side="left")
        k_lbl = tk.Label ( k_param,text="How many ?
",bg=COLOR_CARD,fg=COLOR_TEXT_MAIN,font=("Segoe UI",12,"italic"))
        k_lbl.pack(side="left",padx=15)
        self.k_entry = tk.Entry(
k_param,bg=COLOR_PRIMARY,fg=COLOR_TEXT_MAIN,font=("Segoe
UI",15,"bold"),insertbackground=COLOR_PRIMARY,width=4,bd=0,highlightbackground
=COLOR_BORDER,highlightthickness=1,justify="center")
        # par défaut initialisation de la valeur a K=1
        self.k_entry.insert(0,"1")
        self.k_entry.pack(side="left",padx=5,pady=3)


        #Bouton de déclenchement de la requête de recherche
        self.submit_btttn = tk.Button ( search, text= "SEARCH" ,
bg=COLOR_PRIMARY,fg="white",font=("Segoe
UI",11,"bold"),bd=0,padx=25,pady=8,cursor="hand2",activebackground=COLOR_PRIMA
RY_HOVER,activeforeground="white",command=self.handle_search)
        self.submit_btttn.pack(side="right",padx=6,pady=6)

```

```

        self.submit_btn.bind("<Enter>", lambda e :
self.submit_btn.config(bg=COLOR_PRIMARY_HOVER))
        self.submit_btn.bind("<Leave>", lambda e :
self.submit_btn.config(bg=COLOR_PRIMARY))

        # indicateur textuelle de l'état d'avancement des processus en
arrière-plan
        self.statut_text = tk.StringVar (value="... Connexion With CLIP ...")
        self.statut_lbl =
tk.Label(top_panel, textvariable=self.statut_text, bg=COLOR_BG, fg=COLOR_TEXT_MUT
ED, font=("Segoe UI", 9, "italic"))
        self.statut_lbl.pack(anchor="w", pady=(8, 0))

        #Zone d'affichage dynamique de la grille des photos intégrant une
barre de défilement
        main_content= tk.Frame(self, bg=COLOR_BG, padx=30)
        main_content.pack(fill="both", expand=True)
        self.canvas = tk.Canvas(main_content, bg=COLOR_BG, highlightthickness=0)
        self.scrollbar =
ttk.Scrollbar(main_content, orient="vertical", command=self.canvas.yview)
        self.results_grid = tk.Frame(self.canvas, bg=COLOR_BG)
        self.results_grid.bind("<Configure>", lambda e :
self.canvas.configure(scrollregion=self.canvas.bbox("all")))
        self.canvas.create_window((500, 0), window=self.results_grid, anchor="n")
        self.canvas.configure(yscrollcommand=self.scrollbar.set)
        self.canvas.pack(side="left", fill="both", expand=True)
        self.scrollbar.pack(side="right", fill="y")

        #Liaison de l'événement de défilement à la molette de la souris
        self.canvas.bind_all("<MouseWheel>", self.on_mousewheel)

        # Affichage d'un écran d'accueil par défaut avant l'exécution de la
première requête
        self.placeholder= tk.Label(self.results_grid, text="Everything is
ready`\n Write a description, choose the number of photos and press
Search", bg=COLOR_BG, fg=COLOR_TEXT_MUTED, font=("Segoe UI", 13), justify="center")
        self.placeholder.pack(pady=120, expand=True, anchor="center")

```

```

self.bind("<Configure>",self.on_window_resize)

def on_mousewheel(self,event) :
    self.canvas.yview_scroll(int(-1*(event.delta/120)),"units")

# Gestion d'affichage du texte d'invite (placeholder) dans le champs de la
recherche
def clear_placeholder (self,event) :
    if self.search_entry.get() == "" :
        self.search_entry.delete(0,"end")
        self.search_entry.config(fg=COLOR_TEXT_MAIN)

# Restauration de la valeur par défaut en cas d'absence d'entrée
utilisateur
def restore_placeholder ( self , event ) :
    if not self.search_entry.get().strip() :
        self.search_entry.insert(0,"a dog")
        self.search_entry.config(fg=COLOR_TEXT_MAIN)

# Chargement asynchrone des plongements vectoriels (embeddings) et
instanciation de l'index de recherche FAISS
def load_embeddings (self) :
    def worker() :
        try :
            if not os.path.exists(FICHIER_PTH) :
                self.statut_text.set("Indexation of the photos...")
                # extraction des vecteurs embeddings
                bib.extraction_vecteur_img()
            self.embeddings = torch.load(FICHIER_PTH,weights_only=False)
            # Construction de la matrice par empilement les vecteurs ,
taille : n*512
            self.image_names = list(self.embeddings.keys())
            matrix = torch.stack([self.embeddings[name] for name in
self.image_names])
            #Normalisation L2 de la matrice et conversion au format NumPy
float32 requis par FAISS
            self.dim_embedding = matrix.shape[1]
            matrix = matrix.view(-1,self.dim_embedding)
            matrix_np = matrix.numpy().astype("float32")

```

```

        faiss.normalize_L2(matrix_np)
        # Instanciation de l'index de produit scalaire Flat IP (Inner
Product) pour une recherche exhaustive et optimisée
        self.index = faiss.IndexFlatIP ( self.dim_embedding)
        self.index.add(matrix_np)
        total=len(self.image_names)
        self.statut_text.set(f"system connected - we have {total}
images")

        self.statut_lbl.config(fg=COLOR_PRIMARY)
    except Exception as err :
        self.statut_text.set(f"Err CLIP : {err}")
        self.statut_lbl.config(fg=COLOR_ERROR)

    threading.Thread(target=worker,daemon=True).start()

    # Récupération de la requête de l'utilisateur, validation du paramètre K
et exécution asynchrone de la recherche de similarité
    def handle_search(self) :
        query = self.search_entry.get().strip()
        if not query or query == "" :
            return
        if self.embeddings is None :
            self.statut_text.set("Just wait a few seconds ... ")
            return
        #Validation et conversion sécurisée de la valeur K (Par défaut k=1)
        k_input = self.k_entry.get().strip()
        try :
            k = int(k_input)
            if k <=0 :
                k = 1
        except ValueError :
            k = 1
        self.submit_btn.config(state="disabled",text="Calcul...")
        self.statut_text.set("Matrix sorting and cosine similarity
calculations...")

    def process() :
        try :
            #encodage de texte via CLIP
            text_vector = bib.get_embd_txt(query).view(1,-1)

```

```

        #Normalisation L2 du vecteur requête pour assurer une
équivalence de calcul de la similarité cosinus via le produit scalaire
        text_np = text_vector.numpy().astype("float32")
        faiss.normalize_L2(text_np)
        #extraction des top k images
        k_final = min (k,len(self.image_names))
        top_scores , top_indices = self.index.search(text_np,k_final)
        self.last_results = []
        for score , idx in zip(top_scores[0],top_indices[0]) :
            self.last_results.append({ "name" : self.image_names[idx]
, "score" : score.item()})
        self.after(0,lambda : self.render_grid(query))
    except Exception as err :
        self.after(0,lambda : self.show_error_message(str(err)))

    threading.Thread(target=process,daemon=True).start()

def render_grid ( self,query="") :
    if not self.last_results:
        return
    self.submit_btn.config(state="normal",text="Discover")
    if query :
        self.statut_text.set(f"Results of << {query} >>")
        self.statut_lbl.config(fg=COLOR_PRIMARY)

    #Réinitialisation du conteneur graphique avant l'affichage des
nouveaux résultats
    for child in self.results_grid.wininfo_children() :
        child.destroy()
    self.cashed_photos.clear()

    #Calcul du nombre optimal de colonnes selon la largeur courante de la
fenêtre
    width = self.canvas.wininfo_width()
    if width < 450 :
        columns = 1
    elif width < 780 :
        columns = 2
    elif width < 1100 :

```

```

        columns = 3
    else :
        columns = 4

    for col in range(columns) :

self.results_grid.columnconfigure(col,weight=1,uniform="col_uniform")
        #génération de la galerie des photos à renvoyer
        for index, item in enumerate(self.last_results) :
            filename=item["name"]
            score = item["score"]
            card_frame =
tk.Frame(self.results_grid,bg=COLOR_BG,highlightbackground=COLOR_BORDER,highli
ghtthickness=1,bd=0)
            row = index // columns
            col = index % columns
            card_frame.grid ( row=row ,
column=col,padx=14,pady=16,sticky="nswe")
            card_frame.bind("<Enter>" , lambda e , f = card_frame:
f.config(highlightbackground=COLOR_BORDER_ACTIVE))
            card_frame.bind("<Leave>",lambda e , f = card_frame:
f.config(highlightbackground=COLOR_BORDER) )
            #Traitement d'image : ouverture , redimensionnement et mise en
cache mémoire
            img_path = os.path.join("Images",filename)
            try :
                img = Image.open(img_path).convert("RGB")
                img.thumbnail((255,245),Image.LANCZOS)
                photo = ImageTk.PhotoImage(img)
                self.cashed_photos.append(photo)
                img_lbl = tk.Label(card_frame,image=photo,bg=COLOR_CARD)
                img_lbl.pack(fill="x",padx=8,pady=(12,4))
                img_lbl.bind("<Enter>",lambda e , f=card_frame:
f.config(highlightbackground=COLOR_BORDER_ACTIVE))
                img_lbl.bind("<Leave>",lambda e , f=card_frame:
f.config(highlightbackground=COLOR_BORDER))
            except Exception :
                err_placeholder =tk.Label ( card_frame,text="Image
unattainable",fg=COLOR_ERROR,bg=COLOR_CARD,font=("Segoe UI",10))
                err_placeholder.pack(pady=40)

```

```

        # Affichage du rang et du pourcentage
        pourcentage = int (score*100)
        text_color = COLOR_PRIMARY if score > 0.26 else COLOR_ACCENT
        rang = tk.Label(card_frame,text=f"RANK #{index+1} - Match :
{pourcentage}%",bg=COLOR_ACCENT_BG,fg=text_color,font=("Segoe
UI",9,"bold"),padx=14,pady=6)
        rang.pack(pady=(6,12))

        #Repositionnement automatique du défilement au sommet a chaque
rafraichissement
        self.canvas.yview_moveto(0)

# Affichage de la fenêtre dynamique lors du redimensionnement
def on_window_resize (self,event) :
    if event.widget == self and self.last_results :
        if self.resize_debounce_id is not None :
            self.after_cancel(self.resize_debounce_id)
        self.resize_debounce_id = self.after(150,self.render_grid)

# envoyer les messages d'erreurs
def show_error_message(self,msg) :
    self.submit_btn.config(state="normal",text="Discover")
    self.statut_text.set(f"Error : {msg}")
    self.statut_lbl.config(fg=COLOR_ERROR)

# Point d'entrée principale de l'application et le lancement en loop
if __name__ == "__main__" :
    app = ImageSearchAPP()
    app.mainloop()

```

6.2 La bibliothèque utilisée

On a utilisé une bibliothèque à part afin de minimiser le code principale

```

import os
import torch
from PIL import Image

```

```

from transformers import CLIPProcessor, CLIPModel

#Sélectionnement de GPU ou CPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"on utilise : {device}")

#chargement du modèle CLIP et du processeur
model = CLIPModel.from_pretrained ("openai/clip-vit-base-patch32").to(device)
processor = CLIPProcessor.from_pretrained ("openai/clip-vit-base-patch32")

# extraction des vecteurs embeddings d'une requête
def get_embd_txt ( txt : str ) :

    inputs = processor(    text = [txt] ,
                          return_tensors="pt",
                          padding = True, ).to(device)

    with torch.no_grad() :

        outputs= model.get_text_features(**inputs)
        if hasattr( outputs, "pooler_output" ) :
            vecteur = outputs.pooler_output
        elif hasattr ( outputs,"text_embeds") :
            vecteur = outputs.text_embeds
        else :
            # Normalisation des vecteurs
            vecteur = outputs.cpu()
        vecteur = vecteur / vecteur.norm(dim=-1,keepdim=True)
        return vecteur.cpu().squeeze(0)

# # extraction des vecteurs embeddings des images
def get_embd_img_direct (image) :
    # Initialisation des inputs
    inputs = processor(    images=image,
                          return_tensors="pt",
                          padding = True,    ).to(device)

    with torch.no_grad() :
        outputs= model.get_image_features(inputs["pixel_values"])
        if hasattr( outputs , "pooler_output" ) :
            vecteur = outputs.pooler_output.cpu()
        else :
            # on stock ce vecteur dans image_features et on le normalise

```

```

        vecteur = outputs.cpu()
    vecteur = vecteur / vecteur.norm(dim=-1,keepdim=True)
    return vecteur.cpu().squeeze(0)

# Création d'un fichier pour stocker les données
def extraction_vecteur_img () :
    dossier_images = "Images"

    if not os.path.exists(dossier_images):
        print ("dossier n'existe pas \n")
        exit()

    Liste_images = [f for f in os.listdir(dossier_images) if
f.lower().endswith(('.jpg', '.jpeg', '.jfif', '.webp', '.png'))]

    total_images = len(Liste_images)
    if total_images == 0:
        print("on a pas d'images")
        exit()

    image_vecteurs = {}
    compteur = 0

    print ("extraction des vecteurs pour les images \n")

    with torch.no_grad() :

        for index , nom in enumerate(Liste_images,1) :

            image_path = os.path.join(dossier_images,nom)
            try :
                image = Image.open(image_path).convert("RGB")
                vecteur = get_embd_img_direct(image)
                image_vecteurs[nom]=vecteur
                compteur +=1
                if index % 10 == 0 or index == total_images:
                    print(f"progression : {index}/{total_images} image
analysée : ({(index/total_images)*100:.1f}%)")
            except Exception as e :
                print(f" erreur sur {e} \n")
                continue

```

```

chemin_sauvegarde = "vecteurs_image.pth"

torch.save ( image_vecteurs, chemin_sauvegarde )
print (" les vecteurs sont stocké dans : vecteurs_image.pth")
return image_vecteurs

def afficher_img (nom) :
    image_path = "images"+"/"+nom
    image = Image.open(image_path)
    image.show()

```

7. Conclusion

Ce stage effectué au Centre Borelli (UMR 9010) constitue une opportunité exceptionnelle de concrétiser et d'approfondir les compétences théoriques acquises durant ma Double licence Mathématique et Informatique à l'Université Sorbonne Paris Nord. Le travail réalisé jusqu'à présent a permis de valider avec succès le coeur scientifique de projet : la compréhension du modèle CLIP, l'extraction et le stockage persistant des représentations vectorielles (embeddings), ainsi que l'implémentation des algorithmes de recherche par similarité cosinus.

Le passage d'une recherche séquentielle à une méthode matricielle a mis en évidence l'importance de la structure de données pour optimiser la complexité algorithmique. Les tâches finales du stage qui consistent à intégrer la bibliothèque FAISS pour le passage à l'échelle sur de plus grandes bases et à développer l'interface utilisateur graphique, permettront de transformer ce prototype en une application robuste, fluide et pleinement fonctionnelle.

Sur le plan personnel et professionnel, cette première immersion au sein d'un laboratoire de recherche de premier plan m'apprend à structurer ma démarche scientifique face à des contraintes techniques concrètes. Travailler sur des modèles de fondation en Deep Learning confirme mon vif intérêt pour le Machine Learning et la Data Science, et cette expérience particulièrement stimulante me conforte pleinement dans mon projet de poursuivre mes études vers un cycle d'ingénieur spécialisé dans ces domaines.

8. Bibliographie

8.1. Articles Scientifiques

[1] A. Radford et al., Learning Transferable Visual Models From Natural Language Supervision, ICML, 2021 : <https://arxiv.org/abs/2103.00020>

[2] J. Jia et al., Scaling Up Visual and Vision-Language Representation Learning With Noisy Text Supervision, ICML, 2021: <https://arxiv.org/abs/2102.05918>

[3] R. Girdhar et al., ImageBind : One Embedding Space To Bind Them All, 2023. <https://arxiv.org/abs/2305.05665>

8.2. Documentation Techniques et Liens Web

[4] Hugging Face, “CLIP Documentation and Pre-trained Models” : https://huggingface.co/docs/transformers/v5.9.0/en/model_doc/clip#clip

[5] PyTorch, “PyTorch Documentation (v2.0)” : <https://docs.pytorch.org/docs/2.12/index.html>

[6] IBM Topics, “Qu’est-ce que la similarité cosinus ? “ : <https://www.ibm.com/fr-fr/think/topics/cosine-similarity>

[7] P. Rozwood, “Exploring Image Embeddings with CLIP : A Beginner’s Guide” : <https://medium.com/@prestonrozwood/exploring-image-embeddings-with-clip-a-beginners-guide-e179ee3d3f88>

[8] Tkinter Documentation : <https://docs.python.org/3/library/tkinter.html>

[9] FAISS Documentation : <https://faiss.ai/index.html>

[10] FAISS recherche vectorielle en mémoire : <https://blog.stephane-robert.info/docs/developper/programmation/python/faiss/>